Лабораторная работа №3. Транспонирование матрицы и исследование локальности памяти

Выполнил: Москаленко Дмитрий, группа 11-401. Вариант C (double, N = 4000)

1. Система и условия измерений

Процессор Intel Core i5-12450H: 8 ядер (12 логических процессоров), кеш L1 - 704 КБ (L1d 48 КБ на производительное ядро), L2 - 7 МБ, L3 - 12 МБ. Компилятор GCC 16.2.0 (MSYS2), ключ -O2. Так как таймер тикает по 1 мс, короткие транспонирования в одном замере повторяются до 0,1 с и время делится на число повторов. Результат каждого запуска совпал с transpose_rows (0 несовпадений).

2. Результаты

Таблица 1. Порядок обхода (1 поток)

N	Время rows, с	Время cols, с	BW rows, ГБ/с	BW cols, ГБ/с
256	0,000172	0,000038	6,09	27,90
512	0,000962	0,000704	4,36	5,96
1024	0,006733	0,011889	2,49	1,41
2048	0,0510	0,0690	1,32	0,97
4000	0,1160	0,1340	2,21	1,91

Таблица 2. Размер блока (1 поток, N = 4000, transpose_blocked)

Блок b	4	8	16	32	64	128
Время, с	0,0545	0,0315	0,0290	0,0280	0,0353	0,0318
BW, ГБ/с	4,70	8,13	8,83	9,14	7,25	8,06

Таблица 3. Число потоков (N = 4000, b = b* = 32; S = T(rows, 1 поток) / Tp, T = 0,116 с)

Потоки	Время rows_omp, с	S rows_omp	Время blocked_omp, с	S blocked_omp
1	0,1180	0,98	0,0298	3,90
2	0,0610	1,90	0,0167	6,94
4	0,0360	3,22	0,0151	7,66
8	0,0242	4,79	0,0126	9,24
16	0,0254	4,57	0,0120	9,67

Таблица 4. False sharing (8 потоков, N = 4000, transpose_rows_omp)

Стратегия	schedule(static)	schedule(static, 1)	schedule(static, 8)
Время, с	0,0220	0,0275	0,0242
BW, ГБ/с	11,64	9,31	10,58

Таблица 5. Транспонирование на месте (N = 4000, отдельная серия замеров)

Версия	inplace (1 поток)	inplace_omp (8)	blocked_omp (8)
Время, с	0,0423	0,0110	0,0100
BW, ГБ/с	6,05	23,27	25,60

3. Вывод

transpose_rows и transpose_cols работают с разной скоростью, потому что каждая обходит одну матрицу подряд, а другую - с шагом N элементов: rows прыжками записывает в B, cols прыжками читает из A, а промах при чтении заставляет процессор ждать данные, тогда как запись выполняется через буфер, не останавливая вычисления. Разница становится заметной с N = 1024: две матрицы занимают 16 МБ и уже не помещаются в L3 (12 МБ), и rows обгоняет cols на 15-75 %, а при $N \le 512$ данные лежат в кеше и cols даже быстрее. Найденный b* = 32 связан с кешем L1d: блок A и блок B вместе занимают $2 \cdot 32^2 \cdot 8 = 16$ КБ и помещаются в L1d (48 КБ), поэтому каждая загруженная кеш-линия используется полностью. При b = 64 блоки (64 КБ) в L1d уже не помещаются, а при $b \le 8$ строка блока короче кеш-линии (8 элементов double), и часть загруженных данных пропадает. Ускорение транспонирования ограничено сильнее, чем у интеграла из ЛР2, потому что здесь нет вычислений: каждый элемент один раз читается и один раз записывается, и время определяет пропускная способность памяти, общая для всех ядер. Поэтому rows_omp после 8 потоков перестаёт ускоряться (4,79 → 4,57), а blocked_omp на 8 потоках быстрее себя однопоточного лишь в 2,4 раза и упирается в 20–25 ГБ/с, тогда как интеграл в ЛР2 считался в регистрах и продолжал ускоряться до 16 потоков. schedule(static, 1) медленнее schedule(static) из-за false sharing: соседние строки A обрабатывают разные потоки, поэтому соседние элементы одной строки B - одна кеш-линия из 8 double - записываются разными ядрами, и линия постоянно пересылается между их кешами. При schedule(static) каждый поток пишет в свой непрерывный участок строки B, и такого конфликта нет.

Транспонирование на месте на 8 потоках почти не уступает blocked_omp (0,011 с против 0,010 с) и не требует второй матрицы.